

Tendy: A Multi-Modal Attendance System

Michael Pisman

Comparing Friction & Social Acceptability in Classroom Sensing

EECS 255 | Fall 2025

System Overview

The Problem

Attendance systems balance two opposing forces:

1. **Friction:** How hard is it for the student?
2. **Trust:** How sure are we they are actually there?

The Solution

Tendy is a cross-platform ecosystem (Flutter + Python) that implements **three distinct modalities** to test this trade-off empirically.

Prototype Design: The 3 Conditions

I developed a single app with three "Experiment Modes":

1. Condition A: Dynamic QR (The Baseline)

- *Interaction:* Visual, Active, High Focus.
- *Security:* Rolling TOTP codes prevent static photo sharing.

2. Condition B: Bluetooth LE (The "Magic")

- *Interaction:* Invisible, Passive, Zero Friction.
- *Security:* Proximity-based (RSSI).

3. Condition C: Biometric Selfie (The "Social Cost")

- *Interaction:* High Social Friction, Identity-focused.
- *Security:* Liveness detection + Manual Review.

Implementation: Architecture

Frontend (Flutter)

- **QR:** `mobile_scanner` with a custom "Sliding Window" logic to capture 3 consecutive valid codes.
- **BLE:** `flutter_ble_peripheral` (Advertising) + `flutter_blue_plus` (Scanning).
- **Bio:** `google_mlkit_face_detection` for on-device smile probability.

Backend (FastAPI + MongoDB)

- **Real-time Dashboard:** Polling endpoint `/session/{id}/logs`.
- **Rich Metadata:** Stores raw proofs (RSSI dBm, Scan Intervals, Base64 Images) for HCI analysis.

Implementation: "Rich Metadata"

I don't just log "Present". I log **how** they checked in.

```
{
  "student_id": "student-1",
  "method": "BLE",
  "metadata": {
    "rssi": -68,
    "device_model": "iPhone 15 Pro",
    "os_version": "iOS 17.2"
  }
}
```

- **BLE:** Signal strength maps to physical distance.
- **QR:** Scan count maps to aiming difficulty.
- **Selfie:** Smile probability maps to compliance.

Implementation: Condition A (Dynamic QR)

The "Active" Baseline

- **Mechanism:** Instructor app generates a new QR code every 2 seconds containing `SessionID:TOTP`.
- **Client Logic:**
 - Uses `mobile_scanner` to read the stream.
 - **Sliding Window Algorithm:** Must capture **3 consecutive valid codes** to prove presence over time (approx. 6 seconds).
 - Prevents "Drive-by" scanning or taking a photo of the screen for a friend.
- **Technical Challenge:** Synchronizing the TOTP generation between the instructor's local timer and the server's validation window.

Implementation: Custom TOTP Logic

Time-based One-Time Password (TOTP) is a common 2FA mechanism. It generates short-lived codes based on a shared secret and the current time.

Why not standard TOTP?

Standard TOTP (RFC 6238) uses 30-second windows. This is **too slow** for preventing photo-sharing fraud. I needed a **2-second window** to force real-time presence.

My Solution

1. **Instructor App:** Generates `HMAC-SHA1(Secret, Time / 2s)`.
2. **Student App:** Scans code, extracts TOTP, and buffers it.
3. **Server:** Validates the code against a **±1 window** (allowing for 2-4 seconds of clock drift).
4. **Session Binding:** The QR code embeds the `SessionID` directly, preventing replay attacks across different classes.

Implementation: Condition B (Bluetooth LE)

The "Passive" Magic

- **Mechanism:** Instructor device acts as a BLE Beacon (Peripheral Mode) advertising a unique `SessionUUID`.
- **Client Logic:**
 - Background scanning for the specific UUID.
 - **RSSI Thresholding:** Only submits check-in if signal > -90 dBm.
 - Submits `device_model` and `os_version` to help normalize signal variance.
- **Technical Challenge:** iOS vs. Android BLE stack differences. iOS creates "masqueraded" MAC addresses, requiring UUID-based discovery instead of MAC-based.

Implementation: Condition C (Biometric)

The "Social" Cost

- **Mechanism:** Student enters a 6-digit "Class Code" (prefix of Session ID) to locate the session.
- **Client Logic:**
 - **Liveness Detection:** Uses Google ML Kit to detect a face + **Smile Probability > 0.8**.
 - **Image Capture:** Takes a high-res photo only when the smile is detected.
 - **Upload:** Base64 encodes the image and sends it to the instructor dashboard for manual verification.
- **Technical Challenge:** Balancing image quality (for verification) with upload latency (for UX).

Evaluation: Variables

Independent Variables

- **Check-in Method:** QR Code vs. Bluetooth vs. Biometric.

Dependent Variables

- **Efficiency:** Time to complete check-in (ms).
- **Success Rate:** Percentage of first-try successes.
- **User Satisfaction:** System Usability Scale (SUS) score.
- **Perceived Privacy:** Likert scale rating (1-5).

Control Variables

- **Device:** Same student device for all 3 conditions.
- **Environment:** Same classroom lighting and noise level.
- **Task:** "Check in to the Intro to HCI class."

Evaluation: HCI Metrics

System Usability Scale (SUS)

- I employ the standard 10-item SUS questionnaire to calculate a composite usability score (0-100).
- **Goal:** Determine if the "invisible" BLE method actually results in a higher SUS score than the familiar QR method.

Quantitative Data Analysis

Performance Metrics

- **Student Check-in Time:** Average time from opening the app to successful confirmation.
- **Class Saturation Time:** Time required for 90% of the class to be marked present.
- **Error Rate:** Average number of failed attempts per successful check-in (e.g., "Beacon not found" or "Face not detected").
- **Time-on-Task:** Measured automatically by the app (from "Start" to "Success").
- **Error Rate:** Number of failed attempts (e.g., "Beacon not found" or "Face not detected").
- **Signal Variance (BLE):** Analyzing the standard deviation of RSSI values to determine reliability zones.

Statistical Analysis

- **ANOVA:** I will use a one-way ANOVA to compare the mean completion times across the three conditions.

Qualitative Data Analysis

Post-Task Survey

- **Likert Scale (1-10):**
 - "I felt comfortable using this method."
 - "I trust this method to record my attendance correctly."
 - "I felt my privacy was protected."
- **Open-Ended Questions:**
 - "Which method would you prefer to use every day for a semester?"
 - "Did you feel 'watched' during the selfie check-in?"

The "Creepiness" Factor

- I specifically analyze the qualitative feedback on the Biometric condition to quantify the "Social Cost" of high-assurance attendance.

Evaluation Approach

Status Update

- **System Development:** Complete.
- **User Study:** Scheduled for the next 3 weeks. No data collected yet.

Study Design

- **Method:** Within-Subjects Design.
- **Participants:** N=12 (Simulated Roster).
- **The Roster:** I implemented a "**Mock Roster**" in the app (Student 1 (QR) → Student 1 (BLE) ...) to allow distinct data collection for every condition per user.

Study Procedure

1. Briefing (5 min):

- Participant signs consent form.
- I explain the three technologies (QR, BLE, Bio) without revealing the specific hypotheses.

2. Task Execution (15 min):

- Participant is assigned a random order (e.g., A -> C -> B) to counterbalance learning effects.
- They perform the check-in task for each condition using the specific "Mock Student" identity.

3. Debriefing (5 min):

- Participant completes the Post-Task Survey.
- Semi-structured interview about their experience and preference.

Evaluation: Tasks & Metrics

Condition	Task	Metric (Recorded)
QR Code	Scan 3 valid codes from the screen.	Avg Scan Interval (ms) <i>Cognitive Load</i>
Bluetooth	Select class and wait for auto-connect.	RSSI (dBm) <i>Reliability/Distance</i>
Biometric	Enter code, smile, take photo.	Smile Probability <i>Social Compliance</i>

Future Directions

1. Offline-First Architecture

- Currently relies on real-time server connection.
- **Next:** Sign proofs on-device (Ed25519) and upload when online.

2. Geofencing Integration

- Combine BLE proximity with GPS to prevent "Parking Lot" check-ins.

Discussion

Interpretation

- **Selfie vs. Face ID:** I chose *not* to implement full server-side matching.
- **Why?** The **act** of taking the photo creates the necessary social pressure for honesty.
- **Result:** The "Privacy Tax" (awkwardness) is a feature, not a bug. It prevents fraud without needing expensive GPU servers.

BLE Reliability

- **Issue:** Signal noise (RSSI fluctuation) is significant.
- **Finding:** Device model (iPhone vs Android) drastically affects detection range.

Questions & Answers

Thank you for your attention!

Code available at: github.com/michael-pisman/tendy